

QUIZ 5

Nama : Rahma Auliya Hasanah

NIM : 241080200103

Kelas : Informatika 4/A2

Audit/Mencari Bug Error Dan Perbaiki Script terhadap source code di folder challenge4:

A. Security Vulnerability & AUTHENTICATION

1. Tidak ada Kontrol Unik pada Email Pengguna (users.email)

- Letak masalah: Skema database tabel users pada kolom email VARCHAR(100).
- Rincian: Karena tidak ada aturan UNIQUE, satu email bisa dipakai daftar berkali-kali. Saat login, query database akan bingung memilih data yang mana.
- Perbaikan: tambah aturan UNIQUE di kolom email lewat query agar MySQL otomatis tolak kalau ada email kembar yang mau daftar:
`ALTER TABLE users ADD CONSTRAINT unique_email UNIQUE (email);`

2. Kerentanan IDOR (Insecure Direct Object Reference) akibat Ketidadaan Foreign Key

- Letak masalah: Tabel orders pada kolom user_id dan product_id.
- Rincian: Kolom ini hanya menggunakan INT biasa tanpa ada relasi resmi ke tabel asal. Efeknya, user bisa iseng ganti-ganti ID di URL atau form untuk melihat, mengubah, atau memesan barang pakai akun orang lain tanpa ketahuan.
- Perbaikan: Hubungkan tabelnya dengan aturan FOREIGN KEY agar database otomatis menolak kalau ada input ID palsu atau ID yang tidak terdaftar:
`ALTER TABLE orders
ADD CONSTRAINT fk_orders_user FOREIGN KEY (user_id)
REFERENCES users(id),`

```
ADD CONSTRAINT fk_orders_product FOREIGN KEY
(product_id) REFERENCES products(id);
```

3. Celah SQL Injection (SQLi) Massal akibat Query Manual

- Letak masalah: File login.php, products.php, atau profile.php.
- Rincian: Kalau input dari user langsung digabungkan ke dalam string query SQL, hacker bisa menyisipkan perintah jahat untuk menembus login tanpa password atau bahkan menghapus seluruh database.
- Perbaikan: Jangan menggabungkan variabel input langsung ke query. Wajib pakai metode *Prepared Statements*:

```
// Pakai tanda tanya (?) atau nama parameter (:email) untum
mengunci input
```

```
$stmt = $pdo->prepare('SELECT * FROM users WHERE email =
:email');
```

```
$stmt->execute(['email' => $email]);
```

```
$user = $stmt->fetch();
```

B. Logic Error

1. Manipulasi Harga Belanja Total (orders.total)

- Letak masalah: Logika penanganan input di file order.php saat checkout.
- Rincian: Sistem langsung terima nominal total harga dari form input klien begitu saja. User yang pintar bisa ganti harga barang jadi Rp 1 perak lewat fitur *Inspect Element* di browser sebelum mereka mengklik tombol bayar.
- Perbaikan: Ambil harga asli dari database, lalu hitung totalnya secara mandiri di sisi server backend:

```
// Ambil harga asli barang langsung dari database
```

```
$stmt = $pdo->prepare("SELECT price FROM products WHERE id
= ?");
```

```
$stmt->execute([$productId]);
```

```
$product = $stmt->fetch();
```

```
// Hitung total harga yang valid di sisi server
$totalHargaServer = $product['price'] * $qty;
```

2. Tidak Ada Status Alur Transaksi Pemesanan

- Letak masalah: Skema tabel orders tidak punya kolom status.
- Rincian: karena kolom status tidak ada, sistem akan langsung menganggap transaksi langsung selesai saat data masuk database. Padahal transaksi asli butuh step-step seperti: pending, dibayar, dikirim, selesai, atau batal.
- Perbaikan: Tambah kolom status transaksi dengan tipe data ENUM agar alur belanjanya jelas:

```
ALTER TABLE orders ADD COLUMN status ENUM('pending',  
'paid', 'shipped', 'cancelled') DEFAULT 'pending';
```

3. Pelanggaran Aturan HTTP Method pada Endpoint RESTful API

- Letak masalah: logika penerimaan request di file API seperti order.php atau update_profile.php.
- Rincian: Biasanya developer PHP Native asal-asalan menggunakan metode POST atau GET untuk semua aksi (termasuk update atau hapus data). Ini menyalahi aturan RESTful API yang mewajibkan metode spesifik untuk tiap aksi.
- Perbaikan: Saring request yang masuk menggunakan filter method bawaan PHP agar sesuai standar RESTful:

```
// Kalau buat update data profil, wajibin pakai method PUT:  
if($_SERVER['REQUEST_METHOD'] !== 'PUT') {  
    http_response_code(405);  
    echo json_encode(["message" => "Request harus pakai method  
PUT!"]);  
    exit();  
}
```

C. Performance Issue

1. Lambatnya Operasi `JOIN` akibat Ketiadaan Indeksing Kolom Relasi

- Letak masalah: Kolom orders.user_id dan orders.product_id di database.
- Rincian: Kolom-kolom ini pasti akan sering di-JOIN untuk menampilkan riwayat belanja. Kalau data transaksi sudah puluhan ribu dan kolomnya tidak di-index, database akan lemot banget karena harus mencari datanya secara manual dari atas sampai bawah.
- Perbaikan: Pasang indeks pencarian di kolom relasi agar proses JOIN tabel jadi scepat:

```
CREATE INDEX idx_orders_user_id ON orders(user_id);
```

```
CREATE INDEX idx_orders_product_id ON orders(product_id);
```

2. Bottleneck saat Proses Validasi Login Pengguna

- Letak masalah: Kolom users.email tidak terindeks di database.
- Rincian: Tiap ada yang login di login.php, backend akan mencari baris email user di database. Kalau kolom email tidak di-index, database harus membaca seluruh baris dari awal sampai akhir hanya untuk mencari satu email itu.
- Perbaikan: pasang indeks di kolom email (Catatan: kalau sudah melakukan perbaikan pasang UNIQUE, indeks akan otomatis dibuat sendiri oleh MySQL):

```
CREATE INDEX idx_users_email ON users(email);
```

3. Potensi Kebocoran Sumber Daya Server (DDoS/Spamming API) akibat Tidak Ada Rate Limiting

- Letak masalah: Semua file endpoint API e-commerce.
- Rincian: API yang bisa diakses publik (misal login atau tambah pesanan) mudah diserang pakai bot spammer. Tanpa adanya pembatasan, server bisa langsung down atau terkunci karena disepam jutaan request palsu tiap detik.
- Perbaikan: Pasang codingan rate limiter sederhana untuk membatasi jumlah request maksimal per menit dari IP yang sama:

```
http_response_code(429);
```

```
echo json_encode(["message" => "Kamu nge-spam! Coba lagi nanti
setelah 1 menit."]);
exit();
```

D. Best Practice Violation

1. Tidak Ada Jejak Rekam Waktu Audit Data (Missing Timestamps)

- Letak masalah: Di semua tabel database (users, products, orders).
- Rincian: Tidak ada kolom pencatatan waktu itu menyusahkan. Kita jadi tidak bisa nmelacak kapan user mendaftar, kapan harga produk diedit, atau kapan transaksi belanja itu kejadian.
- Perbaikan: Tambahkan kolom penanda waktu agar rapi dan mudah dilacak saat diaudit:

```
ALTER TABLE orders ADD COLUMN created_at TIMESTAMP
DEFAULT CURRENT_TIMESTAMP;
```

```
ALTER TABLE users ADD COLUMN created_at TIMESTAMP
DEFAULT CURRENT_TIMESTAMP;
```

```
ALTER TABLE products ADD COLUMN created_at TIMESTAMP
DEFAULT CURRENT_TIMESTAMP,
```

```
ADD COLUMN updated_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP;
```

2. Arsitektur Kode Berantakan (PHP Native Monolitik Spageti)

- Letak masalah: Struktur peletakan file aplikasi yang semuanya ditumpuk di root folder.
- Rincian: mencampurkan codingan HTML, logika bisnis PHP, rute API, dan query SQL langsung di dalam satu file akan membuat codingan jadi berantakan (spageti). Susah dibetulkan kalau ada yang error.
- Perbaikan: Rapikan struktur folder proyek pakai konsep MVC (Model, View, Controller) biar codingannya terpisah rapi

3. Kebocoran Pesan Error Database ke Klien (Information Disclosure)

- Letak masalah: File koneksi database db.php atau blok codingan catch error.
- Rincian: Menggunakan perintah seperti `die(mysqli_error())` atau langsung cetak pesan error database asli ke layar itu bahaya banget. Hacker bisa tahu struktur tabel dan nama kolom database dari pesan error yang bocor itu.
- Perbaikan: Tangkap error-nya menggunakan blok try-catch, simpan error aslinya di file log internal rahasia server, lalu tampilkan pesan ramah ke user:

```
try {  
    // Proses query SQL di sini  
} catch (PDOException $e) {  
    // Catat error aslinya secara rahasia ke log server:  
    error_log($e->getMessage());  
    // Tampilkan pesan aman ke user luar:  
    http_response_code(500);  
    echo json_encode(["message" => "Sistem kami lagi bermasalah.  
Coba lagi nanti, ya!"]);  
}
```

E. Bug yang dapat menyebabkan sistem gagal

1. Crash Efek Domino Akibat Penghapusan Data (*Orphaned Records*)

- Letak masalah: Penanganan relasi data saat proses hapus produk.
- Rincian: Kalau admin menghapus produk dari tabel products, data orderan lama yang mencatat produk tersebut bakal pincang. Saat user membuka halaman riwayat belanja, kodingan PHP akan crash (blank putih) karena mencari nama produk kosong yang ID-nya sudah dihapus admin.
- Perbaikan: Perlu dikasih coding cek dulu pakai if-else di PHP-nya sebelum menampilkan data produk agar tidak crash:

// Cek produknya ada atau tidak di database:

```

if (!$productName) {
    $displayItemName = "[Produk ini sudah dihapus oleh Admin]";
} else {
    $displayItemName = $productName;
}

```

2. Kegagalan Decoding Payload JSON Mengakibatkan Internal Server Error (500)

- Letak masalah: Logika pembacaan kiriman data JSON pada RESTful API.
- Rincian: API membaca data kiriman JSON dengan fungsi `json_decode(file_get_contents('php://input'))`. Kalau klien iseng mengirim format JSON rusak (misal kurang tanda koma), fungsi PHP ini akan menghasilkan nilai null dan membuat web langsung crash error 500 saat datanya mau diproses.

- Perbaikan: Validasi dulu hasil konversi JSON-nya, kalau rusak atau menghasilkan null langsung tolak request-nya dengan status 400:

```

$data = json_decode(file_get_contents('php://input'), true);
// Cek format JSON-nya rusak atau tidak:
if (json_last_error() !== JSON_ERROR_NONE || empty($data)) {
    http_response_code(400);
    echo json_encode(["message" => "Format JSON yang kamu kirim rusak!"]);
    exit();
}

```

3. Duplikasi data session bertabrakan saat update profil

- Letak masalah: Logika sinkronisasi data session pada file `update_profile.php`
- Rincian: Saat user ganti nama atau email, sistem akan mengubah isi data session aktif. Kalau struktur data session yang baru dibuat beda format sama yang dibuat saat awal login (misal saat login pakai format objek, saat update pakai format array biasa), halaman lain

yang membaca session tersebut akan langsung mengalami fatal error dan crash total.

- Perbaikan: Buat satu fungsi atau method khusus yang seragam untuk me-refresh data session, jadi format datanya selalu sama di semua halaman dan tidak crash:

// Pakai satu fungsi standar tiap kali mau update data session:

```
function refreshUserSession($userId, $pdo) {  
    $stmt = $pdo->prepare("SELECT id, name, email FROM users  
WHERE id = ?");  
    $stmt->execute([$userId]);  
    $_SESSION['user_data'] = $stmt->fetch(PDO::FETCH_ASSOC);  
}
```